

FreeRTOS OS API

1.Os

1.1.特性

临界区指的是一个访问的公共资源的程序片段，而这些公共资源无法同时被多个线程访问。当一个线程进入临界区时会关闭中断、关调度，而其他线程想进入临界时，会在等待前一个线程退出临界区并恢复中断。

1.2.包含头文件

```
#include <os.h>
```

1.3.接口

```
void os_enter_critical(void)
```

功能：进入临界区

参数：无

返回值：无

注意：

- 1、进入临界区后会直接关闭中断、关调度、退出后恢复中断。
- 2、可以递归调用、enter 与 exit 需对称。
- 3、建议不要在临界区处理太多工作。

```
void os_exit_critical(void)
```

功能：退出临界区

参数：无

返回值：无

```
int os_is_enter_critical(void)
```

功能：判断现在是否进入临界区

参数：无

返回值：

- 1：表示已经进入临界区
- 0：表示没有进入临界区

```
int os_in_handler_mode(void)
```

功能：判断现在是在中断

参数：无

返回值：

- 1：表示已经进入中断
- 0：表示没有进入中断

```
int os_is_runing(void)
```

功能：判断系统是否启动任务调度

参数：无

返回值：

- 1：系统已启动任务调度
- 0：系统没有启动任务调度

```
uint64_t os_get_total_idle_time(void)
```

功能：cpu 进入 idle(空闲状态)的总时间

参数：无

返回值：当前系统空闲时间（单位：微秒）

1.4.实例

例子代码在example/os/thread_test_idle_time_example.c

2.Thread

2.1.特性

线程函数可以创建线程，设置线程的属性以及可以等待与唤醒线程。

2.2.包含头文件

```
#include <os.h>
```

2.3.接口

```
thread_ptr_t thread_create(const char *thread_name, unsigned int stack_size, thread_func_t thread_func, void *data)
```

功能：以默认优先级创建线程

参数：

thread_name	// 线程名称
stack_size	// 线程堆栈空间大小（以字节为单位）
thread_func_t	// 入口函数名
data	// 传入函数的参数

返回值：

- 失败：NULL
- 成功：创建线程句柄（线程控制块的首地址）

```
void thread_delete(thread_ptr_t thread)
```

功能：删除线程

参数：thread // 线程句柄

返回值：无

注意：

- 1、当参数为 NULL 时表示删除当前线程。
- 2、线程执行函数返回时会自动调用 thread_delete 函数进行销毁。

`void thread_set_priority(thread_ptr_t thread, os_priority_id priority)`

功能：设置线程优先级

参数：

```
thread           // 线程句柄
priority         // 线程优先级 ID
/*
OS_priority_idle = 0,    // 最低优先级
OS_priority_low,      // 优先级较低（推荐给运算量大且没有太高时限要求的线程）
OS_priority_normal,    // 创建线程的默认优先级
OS_priority_high,     // 优先级高（推荐给优先级较高的应用程序）
OS_priority_realtime,  // 最高优先级（推荐给需要实时响应的驱动程序）
OS_priority_err,      // 错误优先级
*/
```

返回值：无

`os_priority_id thread_get_priority(thread_ptr_t thread)`

功能：获取线程优先级

参数：thread // 线程句柄

返回值：线程优先级 ID

`thread_ptr_t thread_get_current(void)`

功能：获取当前线程的句柄

参数：无

返回值：线程句柄

`void thread_wait(void)`

功能：线程等待

参数：无

返回值：无

注意：

- 1、不可在中断中使用。
- 2、需要用 thread_wakeup 唤醒线程。

`int thread_wait_timeout(uint32_t ms)`

功能：线程超时等待

参数：ms // 超时时间(单位：毫秒)

返回值：

- 0：未超时
- 1：超时

注意：

- 1、不可在中断中使用。
- 2、需要用 thread_wakeup 唤醒线程。

`void thread_wakeup(thread_ptr_t thread)`

功能：线程唤醒

参数：thread // 线程句柄

返回值：无

注意：在 wakeup 唤醒先于 wait 等待被调用一次或者多次时，wait 等待只会被唤醒一次，详细请看实例。

`void msleep(unsigned int msec)`

功能：休眠（不占用 CPU 资源）

参数：msec // 休眠时间(单位：毫秒)

返回值：无

```
uint64_t thread_get_totalruntime(thread_ptr_t thread)
```

功能：获取线程运行占用 cpu 总时间(单位：微秒)

参数：thread // 线程句柄

返回值：

大于 0：运行时间

0：失败

注意：

- 1、参数填 NULL 默认是当前线程。
- 2、从线程运行开始计时到函数调用结束。

2.3.实例

例子代码在example/os/thread_example.c

3.Thread_waiter

3.1.特性

使用等待的目的是等待线程运行所需的资源条件还没有得到，就会等待其他线程先提供运行的资源与条件再来唤醒等待，一个唤醒只能唤醒一个等待,若唤醒先于等待被调用一次或者多次时，等待只能被唤醒一次。

3.2.包含头文件

```
#include <os.h>
```

3.3.接口

```
#define DEFINE_THREAD_WAITER(waitername)
```

功能：静态方式初始化线程等待

参数：等待变量名

```
void thread_waiter_init(thread_waiter_t *waiter)
```

功能：初始化线程等待

参数：waiter // 等待变量

返回值：无

```
int thread_waiter_wait_timeout(thread_waiter_t *waiter, uint32_t timeout_ms)
```

功能：带超时线程等待

参数：

waiter // 等待变量

timeout_ms // 超时时间（单位：毫秒）

返回值：

-1：超时

0：未超时

注意：设置无限等待可以令 timeout_ms = OS_TIMEOUT_NOT_LIMIT_MS。

```
void thread_waiter_wait(thread_waiter_t *waiter)
```

功能：线程等待

参数：waiter // 等待变量

返回值：无

```
void thread_waiter_wakeup(thread_waiter_t *waiter)
```

功能：唤醒线程等待者

参数：waiter // 等待变量

返回值：无

注意：在 wakeup 唤醒先于 waiter_wait 等待被调用一次或者多次时，waiter_wait 等待只会被唤醒一次，详细请看实例。

3.4.实例

例子代码在example/os/thread_waiter_example.c

4.Mutex

4.1.特性

互斥锁可以保证一段共享的数据操作是完整的。在这共享的数据区域加上互斥锁后，该共享区域在任意时刻只能有一个线程进行访问以达到保护的目的。

4.2.包含头文件

```
#include <os.h>
```

4.3.接口

```
DEFINE_MUTEX(mutexname)
```

功能：静态方式初始化互斥锁

参数：mutexname // 互斥锁变量名

```
DEFINE_MUTEX_RECURSIVE(mutexname)
```

功能：静态方式初始化一个可以递归使用的互斥锁

参数：mutexname // 互斥锁变量名

```
void mutex_init(struct mutex * mutex)
```

功能：初始化一个互斥锁

参数：mutex // 互斥锁变量

返回值：无

注意：此初始化不能递归调用同一把锁。

```
void mutex_init_recursive(struct mutex * mutex)
```

功能：初始化一个可以递归使用的互斥锁

参数：mutex // 互斥锁变量

返回值：无

注意：此初始化可以递归调用同一把锁，但必须在同一线程，递归调用时 lock 与 unlock 需要对称。

```
void mutex_lock(struct mutex * mutex)
```

功能：上锁互斥锁

参数：mutex // 互斥锁变量

返回值：无

注意：

- 1、不能在中断中使用。
- 2、可能会触发系统调度。

```
void mutex_unlock(struct mutex * mutex)
```

功能：解锁互斥锁

参数：mutex // 互斥锁变量

返回值：无

注意：

- 1、不能在中断中使用。
- 2、可能会触发系统调度。

```
int mutex_try_lock(struct mutex * mutex)
```

功能：尝试上锁互斥锁

参数：mutex // 互斥锁变量

返回值：

- 1：表示上锁成功
- 0：表示上锁失败已经上锁不得重复上锁

注意：不能在中断中使用。

```
int mutex_is_locked(struct mutex * mutex)
```

功能：判断该互斥锁是否上锁

参数：mutex // 互斥锁变量

返回值：

- 大于 0：表示已经上锁
- 0：表示未上锁

4.4.实例

例子代码在example/os/mutex_example.c

5.Thread_cond

5.1.特性

使用等待队列的目的是等待线程运行所需的资源条件还没有得到，等待线程就会加入等待队列，需要等待其他线程先提供运行的资源与条件，再调用唤醒函数唤醒所有等待或者唤醒一个等待。

5.2.包含头文件

```
#include <os.h>
```

5.3.接口

`DEFINE_THREADCOND(condname)`

功能：静态初始化一个可用的等待队列

参数：condname //互斥锁变量

`void thread_cond_init(thread_cond_t *cond)`

功能：初始化一个等待队列

参数：cond // 等待队列变量

返回值:无

注意：一般用于线程与线程。

`int thread_cond_wait_timeout(thread_cond_t *cond, struct mutex * mutex, uint32_t timeout_ms)`

功能：阻塞等待队列

参数：

cond // 已经初始化的等待队列
mutex // 互斥锁（不能是递归互斥锁）
timeout_ms // 超时时间（单位：毫秒）

返回值：

0：表示成功激活

-1：表示等待超时

注意：

- 1、不可在临界区和中断中使用。
- 2、在使用线程等待队列时一定要用互斥锁保护。
- 3、设置无限等待可以令 `timeout_ms = THREAD_COND_TIMEOUT_NO_LIMIT_MS`。
- 4、当等待线程优先级不同时，按照优先级顺序插入等待队列。当等待优先级相同时，按照调用的时间顺序插入等待队列。
- 5、当等待时会释放互斥锁，等待结束会重新上锁。
- 6、当在唤醒先于等待被调用时，等待不会被唤醒。

`void thread_cond_wait(thread_cond_t *cond, struct mutex * mutex)`

功能：无限期阻塞等待队列

参数：

cond // 已经初始化的等待队列变量
mutex // 互斥锁（不能是递归互斥锁）

返回值:无

注意：

- 1、不可在临界区中使用和中断中使用。
- 2、当等待线程优先级不同时，按照优先级顺序插入等待队列。当等待优先级相同时，按照调用的时间顺序插入等待队列。
- 3、当等待时会释放互斥锁，等待结束会重新上锁。
- 4、当在唤醒先于等待被调用时，等待不会被唤醒。

`void thread_cond_signal(thread_cond_t *cond)`

功能：唤醒单个正在阻塞的等待队列

参数：cond // 已经初始化的等待队列变量

返回值:无

注意：不推荐在中断使用，因为关键的资源保护需要 mutex 锁配合使用。mutex 锁不能在中断使用导致关键资源就得不到保护。

```
void thread_cond_broadcast(thread_cond_t *cond)
```

功能：唤醒所有正在阻塞的等待队列

参数：cond // 已经初始化的等待队列变量

返回值：无

注意：不推荐在中断使用，因为关键的资源保护需要 mutex 锁配合使用。mutex 锁不能在中断使用导致关键资源就得不到保护。

5.4.实例

例子代码在example/os/thread_cond_example.c

6.Critical_thread_cond

6.1.特性

使用临界区等待队列的目的是等待线程运行所需的资源条件还没有得到，等待线程就会加入等待队列，需要等待其他线程先提供运行的资源与条件，再调用唤醒函数来唤醒等待队列。临界区的等待队列只能在临界区中使用，在进入临界区时中断会被关闭，在 wait 时会被退出临界区，当 wait 完成自动进入临界区，最后退出临界区时会恢复中断。

6.2.包含头文件

```
#include <os.h>
```

6.3.接口

```
#define DEFINE_CRITICAL_THREADCOND(condname)
```

功能：静态初始化临界区等待队列

参数：condname // 等待队列变量

```
void critical_thread_cond_init(critical_thread_cond_t *cond)
```

功能：初始化线程临界区等待队列

参数：cond // 已经初始化的等待队列变量

返回值：无

```
int critical_thread_cond_wait_timeout(critical_thread_cond_t *cond, uint32_t timeout_ms)
```

功能：临界区超时等待

参数：

cond // 已经初始化的等待队列变量

timeout_ms // 超时时间（单位：毫秒）

返回值：

-1：超时

0：未超时

注意：

1、不能在中断中使用、只能在临界区中使用。

2、当等待线程优先级不同时，按照优先级顺序插入等待队列。当等待优先级相同时，按照调用的时间顺序插入等待队列。

3、当等待时会退出临界区，等待结束会重新进入临界区。

4、设置无限等待可以令 timeout_ms = THREAD_COND_TIMEOUT_NO_LIMIT_MS。

5、当在唤醒先于等待被调用时，等待不会被唤醒。

```
void critical_thread_cond_wait(critical_thread_cond_t *cond);
```

功能：临界区等待

参数：cond

// 已经初始化的等待队列变量

返回值：无

注意：

- 1、不能在中断中使用、只能在临界区中使用。
- 2、当等待线程优先级不同时，按照优先级顺序插入等待队列。当等待优先级相同时，按照调用的时间顺序插入等待队列。
- 3、当等待时会退出临界区，等待结束会重新进入临界区。
- 4、当在唤醒先于等待被调用时，等待不会被唤醒。

```
void critical_thread_cond_signal(critical_thread_cond_t *cond);
```

功能：发送信号唤醒临界区一个等待

参数：cond

// 已经初始化的等待队列变量

返回值：无

注意：可以在中断中使用。

```
void critical_thread_cond_broadcast(critical_thread_cond_t *cond);
```

功能：唤醒临界区所有的等待

参数：cond

// 已经初始化的等待队列变量

返回值：无

注意：可以在中断中使用。

6.4.实例

例子代码在example/os/critical_thread_cond_example.c